

2026-2 / IIC2173 - E1 | EnergyShark

Acerca de

EnergyShark fue todo un éxito en su primera ronda de financiación. La idea de la energía como un bien de intercambio entre nodos energéticos fue recibida con amplio entusiasmo: son básicamente pequeños contratos de aceptación de energía.

Para controlar el funcionamiento de la red, se decide crear una prueba de concepto para cada una de las ciudades.

Sobre las ciudades

Cada grupo para esta entrega representará una ciudad, la cual será asignada al azar. Estas ciudades tienen códigos asociados. Las ciudades disponibles son:

```
{
  "cities": [
    { "code": "HGW", "name": "Hogwarts" },
    { "code": "COR", "name": "Coruscant" },
    { "code": "REE", "name": "Re-Estize" },
    { "code": "RAP", "name": "Rapture" },
    { "code": "TAL", "name": "Talca" },
    { "code": "LSN", "name": "Los Santos" },
    { "code": "MTI", "name": "Minas Tirith" },
    { "code": "SPR", "name": "Springfield" },
    { "code": "NNY", "name": "New New York" },
    { "code": "KLD", "name": "King's Landing" },
    { "code": "TAR", "name": "Tar Valon" },
    { "code": "TK3", "name": "Tokyo-3" }
  ]
}
```

Cada ciudad tiene una cola asignada. Para enviar mensajes a esas colas, deben utilizar el binding `city.{cityId}` donde `cityId` es el `code` de una ciudad.

Cada grupo implementará un nodo de energía que cumpla con las especificaciones a continuación.

Formato básico de los mensajes

Cada mensaje, recibido o enviado, tiene al menos:

```
{
  "idpk": "uuid",
  "msgId": "uuid",
  "type": "request",
  "timestamp": "ISO 8601"
}
```

- *msgId* lo determina la entidad que envía y es un UUID aleatorio.
- *idpk* corresponde a lo que se llama una llave de idempotencia, la que asegura que frente a múltiples reenvíos, las operaciones sensibles no se ejecuten más de una vez. Múltiples reintentos del mismo *msgId* deberían tener la misma llave de idempotencia para una misma operación.
 - DEBE diferir del *msgId* puesto que las respuestas harán referencia al *msgId*.
- *type*: identificador de mensaje para alguno de los tipos disponibles.
- *timestamp*: generado al momento de creación del mensaje.

Si un mensaje no puede parsearse o no incluye *msgId*, debe descartarse y registrarse en sus logs (no hay mensaje válido al cual responder). Si el envelope es parseable pero el mensaje no cumple con **al menos** estos 4 campos, o su contenido es inválido, debe responderse con un NACK. Un mensaje bien formado que la central simplemente no puede atender no es un NACK sino un *error* (ver § Errores).

Nota de compatibilidad (protocolo v2): a partir de esta entrega el envelope cambia respecto de la E0 — *msgId* y *timestamp* pasan a ser obligatorios y el contenido viaja en *data* (antes *packageBody*). La central emite todos sus mensajes en este formato desde ahora.

Si ustedes envían el mensaje, deben adjuntar su *cityId* (code):

```
{
  ... //campos comunes msgId, idpk, type, timestamp
  "cityId": "city-code",
  ...
}
```

Los mensajes emitidos por la central no llevan *cityId* — la central no es una ciudad — sino *sender*:

```
{
  ... //campos comunes msgId, idpk, type, timestamp
  "sender": "central",
}
```

```
...  
}
```

Las ciudades se identifican con `cityId` (su `code`); la central se identifica con `sender`. Todo mensaje que la central emite lo lleva.

La propiedad `user_id` al publicar

Al publicar, deben adjuntar la propiedad AMQP `user_id` con el usuario con el que se autenticaron ante el broker: la misma cadena `city.{CODE}` que usan como routing key y como nombre de cola. En la mayoría de las librerías es una opción del `publish`, no un campo del cuerpo del mensaje.

RabbitMQ valida esta propiedad contra la conexión autenticada y rechaza directamente una publicación que declare un `user_id` ajeno — el mensaje ni siquiera llega a la central.

La central verifica además que el `user_id` entregado corresponda al `cityId` declarado en el mensaje. Si no corresponde, el mensaje se rechaza con `reason: "IDENTITY_MISMATCH"`, `code: 403`, y ese rechazo se entrega a la cola de la ciudad que **efectivamente publicó**, no a la que fue nombrada.

La ausencia de `user_id` también se rechaza, con el mismo `reason` y el mismo `code`. RabbitMQ solo valida la propiedad cuando está presente, de modo que aceptar su ausencia dejaría la verificación trivialmente evitable. Un cliente que asuma que la regla solo aplica cuando la propiedad viaja va a dejar de funcionar; adjúntenla siempre.

Casi siempre este error significa una configuración copiada de otro equipo, no un ataque.

ACK y NACK

Si un servicio envía un mensaje a otro servicio, el receptor debe hacer un `acknowledge` del mensaje a la cola del servicio emisor. Este debe verse así:

```
{  
  ...  
  "type": "ack",  
  "data": {  
    "target": "uuid" //msgId del mensaje respondido  
  }  
  ...  
}
```

Si el servicio no respeta el funcionamiento del otro servicio enviando un mensaje malformado, se responderá con un Not ACK, NACK:

```

{
  ...
  "type": "nack",
  "reason": "MALFORMED_MESSAGE",
  "code": 422, //algún código prestado de HTTP
  ...
  "data": {
    "target": "uuid", //msgId del mensaje respondido
    "message": "quantity must be a positive number"
  }
  ...
}

```

`reason` y `code` van en el **nivel superior** del envelope, como hermanos de `type`. Dentro de `data` viajan dos campos adicionales:

- `data.message` — texto humano describiendo la falla.
- `data.cycleId` — el `cycleId` que ustedes enviaron, devuelto tal cual, para que puedan aparear el rechazo con el mensaje que lo causó. Solo aparece cuando el mensaje rechazado traía uno.

El `msgId` de una respuesta (`ack`, `nack`, `error`, `give`, `take`, `transfer`) es **siempre un UUID nuevo y aleatorio**, generado por quien responde — la misma regla que ya rige para cualquier `msgId`. La referencia al mensaje que se está respondiendo viaja en `data.target`, nunca reutilizando el `msgId` original.

Estos son todos los `reason` que puede llevar un NACK, con su `code`:

<code>reason</code>	<code>code</code>
<code>MALFORMED_MESSAGE</code>	422
<code>UNKNOWN_TYPE</code>	400
<code>IDPK_EQUALS_MSGID</code>	422
<code>IDENTITY_MISMATCH</code>	403

No se esperan acciones luego de un NACK, más que el reenvío del mensaje correctamente formado. Será decisión del servicio que recibe el NACK qué hará con este.

No se hacen ACKs de ACKs ni NACKs de NACKs para evitar un loop de exceso de cortesía.

Un ACK solo confirma la **recepción** de un mensaje; no implica aceptación de su contenido (e.g., el ACK de una propuesta de negociación no es la aceptación de la propuesta — esa es la confirmación).

Errores

Un NACK dice "no acepto este mensaje": está malformado, su tipo no existe, su `idpk` es inválido, o su identidad no corresponde. Es un rechazo de **recepción**.

Distinto es el caso en que el mensaje está bien formado y correctamente dirigido, pero la central no puede actuar sobre él: el ciclo que nombra ya venció, o la propuesta excede un límite. Ahí el mensaje fue recibido y entendido; lo que se rechaza es la **operación**. Para eso existe el tipo `error` :

```
{
  ...
  "type": "error",
  "cycleId": "cycle-9431",
  "reason": "OVER_CAPACITY",
  "code": 409, //algún código prestado de HTTP
  "data": {
    "target": "uuid", //msgId del mensaje que causó el error
    "message": "give de 900 excede la capacidad vendible restante",
    "spare": 320
  }
  ...
}
```

`reason` y `code` van en el **nivel superior** del envelope, igual que en un NACK. Dentro de `data` viajan `target` , el texto humano `message` , y —cuando corresponde— el dato legible por máquina que les permite corregir el intento: `data.cap` en un `PRICE_ABOVE_CAP` , `data.spare` en un `OVER_CAPACITY` .

Los `reason` que viajan como `error` son:

reason	code	Cuándo
CYCLE_UNKNOWN	404	el <code>cycleId</code> nombrado no existe
CYCLE_EXPIRED	410	la ventana de negociación de ese ciclo ya cerró
PRICE_ABOVE_CAP	422	el precio ofertado supera el tope del ciclo
OVER_CAPACITY	409	el <code>give</code> excede la energía vendible restante

| `REPORT_TOO_EARLY` | 425 | el reporte llegó antes de que abriera el periodo de cierre de la ventana de negociación |

Como el mensaje sí fue recibido, un `error` **siempre va precedido de un `ack`**. Un NACK, en cambio, nunca lleva `ack` previo: no hubo nada que aceptar. Esa es la forma más rápida de distinguir los dos casos desde el cliente.

No se hacen ACKs de errores, ni errores de errores, por la misma razón que no se hacen ACKs de ACKs.

Sobre el presupuesto

Cada ciudad genera cierta cantidad de energía y demanda cierta cantidad de energía. Esta generación de energía varía a lo largo del día puesto que proviene de diversas fuentes, así como el consumo varía según las demandas de los ciudadanos.

Todas las ciudades funcionan con un ciclo de 2 horas:

- Durante los últimos 20 minutos de cada ciclo se abre la **ventana de negociación** del ciclo siguiente. Al inicio de esta ventana, las ciudades reciben su estado para las 2 horas siguientes; en este espacio, todas las ciudades preparan sus balances de energía y tablas de transmisión. De no recibirlas, deben solicitarlas. Actualmente la relación es exclusivamente con la central.
- 5 minutos antes del cierre de la ventana de negociación, la ciudad debe reportar a la central su budget y balance energético.
- Al cerrar la ventana, comienzan las 2 horas de consumo de lo acordado.

Así, se vería del siguiente modo:

```
Consumo:      | -----2h----- | -----2h----- | -----2h----- | -----2h----- |
Negociacion:   | ----- | 20m | ----- | 20m | ----- | 20m | ----- | 20m |
```

Notar el solape: la negociación del ciclo siguiente ocurre durante la última parte del consumo del ciclo actual.

Además de la energía generada, cada ciudad recibe en fase de negociación un balance de presupuesto para las siguientes 2 horas, el cual determina cuántos fondos hay para gastar.

Cada ciclo de negociación activo tendrá un id asociado, presente en todos los mensajes ligados al ciclo:

```
{
  ...
  "cycleId": "cycle-9431",
  ...
}
```

El `cycleId` es un identificador ordinal legible (`cycle-{n}`), no un UUID. Trátenlo como una cadena opaca: no deriven significado de su valor más allá de compararlo por igualdad.

La central responderá con un mensaje `error` (`CYCLE_EXPIRED` o `CYCLE_UNKNOWN` , siempre precedido de su `ack`) los mensajes de ciclo cuyo `cycleId` esté vencido o no exista. Las peticiones directas (`type: "request"`) no están ligadas a un ciclo.

Sobre el periodo de negociación (span-n)

Al inicio recibirán la capacidad actual de energía de su ciudad para las siguientes 2 horas en el siguiente formato. Este status-statement es válido por 20 minutos:

```
{
  ...
  "type": "status-statement",
  "cycleId": "cycle-9431",
  "data": {
    "energy": {
      "generationCapacity": 1234512,
      "consumption": 1444121,
      "generationCost": 210
    },
    "validUntil": "2026-09-01T14:20:00Z"
  },
  ...
}
```

- La energía generada y consumida está expresada en kilowatt-hora (kWh).
- El costo está expresado en créditos.

Además, se les enviará un mensaje para transferirles fondos:

```
{
  ...
  "type": "transfer",
  "cycleId": "cycle-9431",
  "data": {
    "quantity": 508145 //podría ser otro número
  },
  ...
}
```

Nótese que en cada iteración pueden suceder varios escenarios:

- Puede sobrar budget.
- Puede sobrar energía. Esta energía puede ser vendida a la central a un premium de 5% del precio de su propia producción.
- Puede no alcanzar el presupuesto para la energía demandada.

No pueden quedar sin cubrir su demanda de energía. Pueden quedar con budget negativo por ahora. El budget se traspasa entre ciclos. La energía, en cambio, no se almacena: el excedente no vendido se pierde al cierre del ciclo.

En esta entrega, el ledger (budget y balance energético) es mantenido por cada nodo de ciudad: ustedes son la fuente de verdad de su propio estado, y lo reportan a la central al cierre de cada ciclo mediante el `negotiation-report`. En entregas futuras, el manejo del presupuesto se centralizará en un servicio dedicado.

Sobre la negociación

Cuando les interese una negociación, deben enviar el siguiente mensaje de propuesta:

```
{
  ...
  "type": "negotiation-proposal",
  "cycleId": "cycle-9431",
  "data": {
    "direction": "take", // "give" para entregar, "take" para comprar
    "quantity": 2024,
    "pricePerEnergy": 215
  },
  ...
}
```

Cómo se fija el precio

La central fija **ambos** precios. El `pricePerEnergy` que ustedes envían en la propuesta es un **techo**, no el precio de la operación.

- Un `take` (compran energía) se liquida al `generationCost` de su ciudad para ese ciclo.
- Un `give` (venden energía) se liquida a $1.05 \times \text{generationCost}$ — el premium de 5% ya mencionado en § Sobre el presupuesto.
- Ambos números son derivables por ustedes desde su propio `status-statement`, de modo que todo pago es verificable de su lado sin preguntarle nada a la central.

- Un precio ofertado **por debajo** del precio de la central no reduce lo que la central paga ni cobra. Hay un solo precio por dirección por ciclo.
- El redondeo es: `pricePerEnergy = round2(1.05 × generationCost)` y luego `quantity = round2(energy × pricePerEnergy)`. Se redondea el precio unitario **primero** y el total se calcula a partir del precio ya redondeado, de modo que multiplicar los dos números publicados reproduce exactamente el total de la central.

El tope de la oferta

Una propuesta cuyo `pricePerEnergy` supere `round2(1.05 × generationCost)` es rechazada con un `error, reason: "PRICE_ABOVE_CAP", code: 422`.

- El tope es **el mismo número en ambas direcciones** y **no se compone**: una propuesta `give` que oferte exactamente el precio de venta nunca es rechazada por este motivo.
- El `error` incluye el tope en `data.cap`, como campo legible por máquina y no solo como texto humano. No revela información nueva: la ciudad ya conoce su `generationCost`.

```
{
  ...
  "type": "nack",
  "reason": "PRICE_ABOVE_CAP",
  "code": 422,
  "data": {
    "target": "uuid", //msgId de la propuesta rechazada
    "message": "bid 230 exceeds the cap of 220.5 for cycle-9431",
    "cap": 220.5,
    "cycleId": "..." //el cycleId que ustedes enviaron, tal cual
  },
  ...
}
```

Del mismo modo, un `give` que exceda el máximo vendible (ver § Give) es rechazado con `reason: "OVER_CAPACITY", code: 409`, y ese `error` devuelve en `data.spare` la energía vendible que les queda en el ciclo. Tampoco revela información nueva: `spare` se deriva de su propio `status-statement` menos lo que ya vendieron en ese ciclo.

Confirmaciones

Give

Si es una operación de salida de energía ("give": entregan energía y les pagan por ella), la central enviará dentro de 30 segundos el siguiente mensaje:

```
{
  ...
  "type": "give",
  "cycleId": "cycle-9431",
  "data": {
    "target": "uuid", //msgId del mensaje de la propuesta
    "energy": 2024,
    "pricePerEnergy": 220.5
  },
  ...
}
```

No pueden vender más energía de la que produjeron: el máximo vendible en un ciclo es `max(0, generationCapacity - consumption)`. La central rechazará con un `error ( reason: "OVER_CAPACITY" )` las propuestas give que excedan este límite.

Take

Si es una operación de entrada de energía ("take": reciben energía y pagan por ella), la central enviará dentro de 30 segundos el siguiente mensaje:

```
{
  ...
  "type": "take",
  "cycleId": "cycle-9431",
  "data": {
    "target": "uuid", //msgId del mensaje de la propuesta
    "energy": 2024,
    "pricePerEnergy": 210
  },
  ...
}
```

Pago

Finalmente, la parte que paga emite una transferencia:

```
{
  ...
}
```

```

    "type": "transfer",
    "cycleId": "cycle-9431",
    "data": {
        "becauseOf": "uuid", //msgId del mensaje de confirmación
        "quantity": 435160 //monto acordado: energy * pricePerEnergy
    },
    ...
}

```

Donde *quantity* es el monto acordado. Si ustedes son la parte que paga, deben descontarlo de sus fondos.

Si ustedes son la parte que recibe el pago, deben esperar la transferencia de la central dentro de los 30 segundos posteriores a la confirmación. Si no llega, asuman que no hubo operación real y reintenten la operación con el mismo "idpk".

Sobre reportar a la central

El ciclo se cierra con un mensaje de reporte a la central. Ese reporte se acepta **sólo dentro del periodo de cierre**, que son los últimos minutos de la ventana de negociación —en producción, los últimos 5 minutos; la duración exacta es un valor configurado del despliegue—. Un reporte enviado antes de que ese periodo abra **no queda guardado para después: se rechaza** con `REPORT_TOO_EARLY` (422), que viaja como `error` y por lo tanto va precedido de su `ack`, y trae en `data.opensAt` el instante ISO-8601 desde el cual pueden reintentarlo. Reintenten en ese instante, no de inmediato.

El reporte es la posición de **cierre** de la ciudad para ese ciclo; por eso se acepta en una ventana angosta al final y no en cualquier momento del periodo de negociación.

```

{
    ...
    "type": "negotiation-report",
    "cycleId": "cycle-9431",
    "data": {
        "budgetBalance": 131212,
        "energyBalance": 1232
    },
    ...
}

```

El `cycleId` es obligatorio en este mensaje: un `negotiation-report` que no nombre ciclo se rechaza con `MALFORMED_MESSAGE` (422).

Si llega tarde, no sirve. Un reporte que llega después de que la ventana de negociación cerró se rechaza con `CYCLE_EXPIRED` (410) y **no queda registrado**: para la central ese ciclo quedó sin reporte, y la multa por no haber reportado se aplica igual. Reintentar después del cierre de la ventana no les compra nada.

Sí pueden corregir, mientras el periodo de cierre siga abierto. Un segundo `negotiation-report` para el mismo ciclo, enviado dentro del periodo de cierre, reemplaza al primero: el último es el reporte de la ciudad para ese ciclo. No confundan esto con un reenvío. Un reenvío lleva el **mismo** `idpk` —es la misma operación— y no cambia nada; una corrección es un mensaje nuevo, con un `idpk` nuevo, y sí reemplaza al anterior.

Sólo deben reporte por un ciclo en el que la central les habló. La obligación de reportar el ciclo N existe únicamente si recibieron los mensajes de apertura de ventana de ese ciclo. Si la central no alcanzó a contactarlos en un ciclo, no deben reporte por ese ciclo y no se les multa por él.

No entregar este reporte incurre en una multa de 50000 créditos en su siguiente budget recibido, y esa multa llega descontada y declarada en el `penalty` del `transfer` de ese siguiente ciclo (§ [Sobre el periodo de negociación \(span-n\)](#)).

Sobre la transmisión de energía

Cada ciudad está conectada a otra mediante rutas. Estas rutas determinan cómo pueden enviar energía a otras ciudades y tienen un costo dinámico dirigido por pérdidas de energía (e.g. calor, distancia, método de transmisión, etc.).

Cada cierto tiempo obtendrán una tabla de distancias como esta para su ciudad. Distancia en metros, costo de transporte en `créditos/(kWh*km)` :

```
{
  ...
  "cityId": "COR",
  "type": "distance-table",
  "data": {
    "distances": {
      "HGW": {
        "distance": 62763183,
        "transportCost": 0.0034,
        "enabled": true
      },
      ...
      "TAR": {
        "distance": 94306517,
```

```

        "transportCost": 0.0013,
        "enabled": true
    }
},
...
}

```

A modo de referencia: enviar 1 kWh a una ciudad a 62.763 km con `transportCost` 0.0034 costaría $\approx$ 213 créditos — comparable al precio de la energía misma.

Estas distancias cambiarán con el tiempo. Recibirán un mensaje con el tipo *"distance-table"* en sus colas cada cierto tiempo con los nuevos valores que deberán usar.

En esta entrega no transmitirán a otras ciudades; la tabla solo se recibe, se persiste y se visualiza.

Negociando bolsas de energía con la central

Cada cierto tiempo la central les enviará una petición para solicitar o entregar energía. Por ahora, deben tomarla directamente:

```

{
    ...
    "type": "demand-statement",
    "cycleId": "cycle-9431",
    "data": {
        "balance": {
            "quantity": 1500, //podría ser negativo
            "valuePerKwh": 215
        }
    },
    ...
}

```

Al recibir un `demand-statement` deben hacer ACK y aplicarlo directamente sobre su ledger local:

- `quantity` **positivo**: la central les entrega energía. Su balance energético aumenta en `quantity` y su budget disminuye en `quantity * valuePerKwh`.
- `quantity` **negativo**: la central retira energía. Su balance energético disminuye en `|quantity|` y su budget aumenta en `|quantity| * valuePerKwh`.

Este mensaje no lleva confirmaciones ni transferencias: el efecto es inmediato y debe quedar reflejado en el `negotiation-report` del ciclo.

- Notar que su balance interno puede salir negativo si suceden muchas operaciones desfavorables.
- No confundir con el evento `demand-set` de la EO: aquel informa niveles de demanda; `demand-statement` es una orden de intercambio de la central.

Peticiones directas

Pueden solicitar los mensajes emitidos por la central directamente, aunque se sugiere no sobrecargar este mecanismo: no cambiará si lo piden repetidamente para un ciclo dado.

```
{
  ...
  "type": "request",
  "data": {
    "ask": "XXXX" //e.g. distance-table, status-statement
  },
  ...
}
```

Requisitos

Esenciales (gate de corrección)

Su entrega se corrige **solo si cumple todos** los siguientes puntos. No otorgan puntaje: son condición de entrada. De faltar alguno, la nota máxima de la entrega es 4.0.

- **G01:** La aplicación corre completa en la nube (EC2 free tier). No se corregirán entregas locales ni caídas al momento de la revisión.
- **G02:** El nodo consume su cola `city.{cityId}` y responde ACK/NACK según las reglas de la mecánica.
- **G03:** El ciclo básico opera de forma autónoma: se aplica el `status-statement` y se emite el `negotiation-report` dentro de la ventana, en todos los ciclos, sin intervención manual.
- **G04:** Backend y frontend separados: API con respuestas JSON + SPA (React o Vue con build estático), en **dos repositorios distintos**.
- **G05:** Backend y frontend servidos por HTTPS.

- **G06:** Servicio de autenticación/autorización operativo (Auth0 recomendado, o Cognito) con setup JWK estándar.
- **G07:** Monitoreo SaaS instalado (New Relic recomendado por su free tier).
- **G08:** Budget alerts configuradas en la cuenta AWS del grupo.

RF (16 pts)

- **RF01 (4 pts):** Vista de historial de ciclos. Para cada ciclo debe mostrarse al menos: `cycleId`, el `status-statement` recibido, los fondos recibidos por `transfer`, los `demand-statement` aplicados, las negociaciones voluntarias realizadas, el `negotiation-report` enviado y los balances finales (budget y energía). Debe poder identificarse claramente la última operación aplicada en cada ciclo.
- **RF02 (2 pts):** Vista de conectividad: la `distance-table` vigente (destino, distancia, `transportCost`, `enabled`), actualizada cuando la central publique cambios.
- **RF03 (6 pts):** Ciclo de negociación completo operando de forma autónoma: aplicación de `status-statement`, `transfer` y `demand-statement` (con su convención de signos) sobre el ledger local, flujo voluntario `negotiation-proposal` → confirmación → pago con sus timeouts de 30 segundos y reintento con el mismo `idpk`, y emisión del `negotiation-report` dentro de la ventana.
- **RF04 (2 pts):** Interfaz de administración para negociaciones voluntarias: crear una propuesta, seguir su confirmación y pago, y ver el historial con estado final (confirmada, pagada, expirada por timeout).
- **RF05 (2 pts):** Registro consultable de mensajes duplicados y de mensajes descartados o respondidos con NACK. Los duplicados no deben re-aplicarse al ledger. *(Esta es la evidencia que la demo va a revisar.)*

Características arquitectónicas (16 pts)

Prescritas (12 pts)

- **RNF01 (3 pts):** API detrás de AWS API Gateway (REST o HTTP) con subdominio propio (e.g. `https://api.miapp.com`) y CORS correctamente configurado.
- **RNF02 (2 pts):** API Gateway autentica contra su servicio de auth antes de pasar la request (custom authorizer si usan tipo REST).
- **RNF03 (2 pts):** Frontend desplegado en S3 con distribución CloudFront.
- **RNF04 (2 pts):** Deploy del backend vía ECR sobre EC2, con docker compose optimizado para producción.
- **RNF05 (3 pts):** Monitoreo con APM básico (requests visibles en el controller HTTP) y monitoreo de infraestructura habilitado.

Decisiones de arquitectura (4 ptos)

Para las siguientes tres áreas, el enunciado exige la **propiedad**; el **mecanismo** lo decide su grupo y lo justifica en un ADR (ver RDOC01). No hay una respuesta única correcta: se evalúa la decisión y su defensa.

- **AD1 – Topología del consumo:** cómo estructuran el o los servicios que consumen el broker y alimentan al resto del sistema. *Propiedad exigida:* la caída del broker no bota la API; la reconexión es automática y no se pierden mensajes ya recibidos.
- **AD2 – Persistencia del ledger:** modelo y motor de datos (snapshot vs event log, SQL vs NoSQL). *Propiedad exigida:* el estado del ledger de cualquier ciclo pasado es reconstruible y explicable desde lo persistido.
- **AD3 – Manejo de timeouts de negociación:** cómo implementan los 30 segundos y el reintento con el mismo `idpk`. *Propiedad exigida:* una operación reintentada nunca se aplica dos veces.

Decisiones y documentación (14 ptos)

Todos los documentos van en `/docs` de su repositorio, según los formatos del template repo entregado por el curso.

- **RDOC01 (6 ptos):** Los tres ADRs (AD1–AD3), con alternativas consideradas y tradeoffs, escritos **antes de implementar** la decisión: el commit del ADR debe preceder en el historial git a los commits de implementación correspondientes. Un ADR revisado después de la demo con un post-mortem honesto puntúa completo; una decisión que nunca fue examinada contra la realidad, no.
- **RDOC02 (3 ptos):** Artefactos de proceso: spec de la entrega al abrir la ventana de desarrollo, plan de milestones, y AI logs acordes al uso real de su grupo (o declaración explícita de no-uso), en el formato del template.
- **RDOC03 (3 ptos):** Diagrama UML de componentes con explicación; pasos para replicar el flujo de monitoreo; cómo correr la aplicación en ambiente local (`.env` de ejemplo, instalación, comandos).
- **RDOC04 (2 ptos):** Repo de contratos a nivel de organización (schemas de los mensajes de la mecánica + el OpenAPI de su propia API) y un archivo de contexto compartido (`CLAUDE.md` / `agents.md` org-level) referenciado por ambos repositorios.
-

Demo de comportamiento (14 ptos)

La demo en vivo con su ayudante incluye un segmento evaluado de **anomalías inducidas**: su ayudante provocará **dos situaciones de la siguiente lista**, en vivo, sobre su sistema corriendo. La lista es pública; cuáles les tocarán, en qué momento y con qué contenido, no.

1. **Reenvío duplicado** de un mensaje sensible (mismo `idpk`) → el ledger no cambia dos veces y el duplicado queda registrado (RF05).
2. **Mensaje malformado** o sin `msgId` → NACK o descarte según la regla de la mecánica, sin caída del servicio.
3. **Corte breve de conexión al broker** → reconexión sin intervención manual; la API sigue sirviendo lo ya persistido.
4. **Caída de un contenedor** → se levanta de nuevo; el ciclo en curso no pierde su `negotiation-report`.
5. **Lectura de su propio monitoreo** → explicar qué se observa en el APM y en infraestructura durante la anomalía provocada.
Se evalúa el **comportamiento observado del sistema y la explicación del grupo**: qué pasó, por qué, y dónde vive eso en su código y arquitectura. No se miden números en esta entrega. Las herramientas de AI que su grupo use normalmente están permitidas y bienvenidas durante la demo.

Además, cada integrante tendrá una **defensa individual breve** (~5 minutos): su ayudante elegirá un archivo, un flujo o un panel de su dashboard, y las preguntas serán del tipo *¿por qué existe esto?*, *¿qué se rompe si desaparece?* (ver Evaluación).

Meta: sobre la entrega

- NO se revisarán entregas que no estén en la nube bajo ningún concepto.
- Deben tener los requisitos no variables de la E0 operativos para esta entrega.
- Creen sus repositorios e integren a su ayudante con acceso de administrador.
- Al final de la entrega, agenden con su ayudante la demo en vivo (incluye el segmento de anomalías y la defensa individual).
- Está prohibido subir `.env` (5 décimas de descuento) o `.pem` (no se corrige la entrega) al repositorio.
- Aplica la política de abuso de mensajería del curso (loops infinitos o mensajes deliberadamente maliciosos → corrección en 8 horas, ban escalonado del broker).

Del uso de AI

- Tab completion y consultas por chat: libres.
- Todo grupo entrega en `/docs` sus **AI logs acordes al uso real** (prompts y flujos relevantes, por integrante, formato del template) **o una declaración explícita de no-uso**. No hay notificación previa al ayudante.
- Si usan programación agéntica (una AI escribiendo código directamente en el codebase), los artefactos de proceso del template (spec, milestones, ADRs, logs, flujo de PRs con revisión de al

menos 2 compañeros, main protegido) son obligatorios. Artefactos faltantes descuentan los puntos de RDOC correspondientes – **no anulan la entrega**.

- Uso de AI no reflejado en los logs detectado en la demo o corrección se trata según el reglamento del curso.
- Todos los miembros deben estar preparados para responder preguntas sobre todo el codebase.

Puntaje

Atraso

Para esta entrega se les descontará 0.5 puntos en la nota máxima por horas Fibonacci con F1 = 6 y F2 = 6.

Se considerará como atraso cualquier modificación en features o implementación que tenga que ver solo con lo que se pide en esta entrega.

Fibonacci	Hora	Nota maxima
6	0:01 - 5:59	6.5
6	6:00 - 11:59	6
12	12:00 - 23:59	5
18	24:00 - 41:59	4.5
30	42:00 - 71:59	4
...	72:00 en adelante	1

Grupal

La nota se calcula como:

$$E_{1\text{grupal}} = 1 + E_{1\text{RF}} + E_{1\text{RNF}} + E_{1\text{RDOC}}$$

Siendo $E_{1\text{RF}}$ el puntaje ponderado de los requisitos funcionales, y $E_{1\text{RNF}}$ el correspondiente a los requisitos no funcionales y $E_{1\text{RDOC}}$ de la documentación.

Individual

Segun el programa del curso⁵, esto se evalua como:

$$E_1 = 1 + ((E_{1\text{ grupal}} - 1) * F_g)$$

Donde F_g es un factor de coevaluación asignado por el grupo que va de 0.25 a 1.25. Para esto se enviará un form de coevaluación donde cada integrante deberá evaluar a sus compañeros de grupo con una puntuación entre 1 y 5.

Puntaje	factor
1	0.25
2	0.5
3	0.75
4	1
5	1.25

Si añaden puntos a un compañero, significa que alguien trabajó menos, por lo que los puntos deben salir de otra evaluación

De no realizar la coevaluación, asumiremos que se le entregó el mismo puntaje de coevaluación a cada integrante, es decir 4 puntos. También si la coevaluación no suma bien los puntos, se considerará que no la respondieron.